

特征点提取与光流法追踪

2026 年 8 月

1 引言：数学的美妙巧合

在计算机视觉中，我们要完成两件事：一是找出画面里最容易追踪的“特征点”（角点提取），二是计算这些点在两帧之间的移动速度（光流法）。在调用封装接口时，这看似是两个毫不相干的函数。但当我们今天用 C++ 纯手工实现它们底层时，你会发现一个令人惊叹的数学巧合：这两个算法的核心，竟然是求解完全相同的同一个矩阵！

2 核心基石：结构张量矩阵 (Structure Tensor)

无论是角点检测，还是光流法，第一步都是计算图像的微积分导数：水平偏导 I_x 和垂直偏导 I_y 。在一个像素点周围的 $W \times W$ 邻域内，我们构建一个 2×2 的矩阵 M ，称为结构张量：

$$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \quad (1)$$

这便是连接感知与运动的终极钥匙。

3 特征点提取 (Shi-Tomasi)：求矩阵的特征值

3.1 原理解析

什么样的点才是“好点”（角点）？如果一个点在图像中是平坦的墙面，矩阵 M 的值全接近 0；如果是一条垂直的边缘，它在 Y 方向没有变化。在线性代数中，矩阵的特征值 (Eigenvalues) 代表了数据分布在主成分上的方差。Shi 和 Tomasi 提出：只要矩阵 M 的两个特征值 λ_1, λ_2 中的极小值足够大，说明这个点在任何方向都有剧烈梯度，它必定是一个角点！

3.2 C++ 硬核求解

对于 2×2 矩阵 $M = \begin{bmatrix} a & c \\ c & b \end{bmatrix}$, 求其特征值即解一元二次方程 $\lambda^2 - (a+b)\lambda + (ab - c^2) = 0$ 。我们在代码中直接手写求根公式得出最小特征值:

```
1 float a = Sxx.at<float>(y, x); // ΣIxx
2 float b = Syy.at<float>(y, x); // ΣIyy
3 float c = Sxy.at<float>(y, x); // ΣIxy
4
5 // 计算最小特征值 _min
6 float lambda_min = (a + b - sqrt((a - b)*(a - b) + 4 * c * c)) / 2.0f;
;
```

通过设定阈值并进行非极大值抑制（找出局部最亮的点），我们就用纯数学抠出了视野中所有的角点。

4 光流法 (Lucas-Kanade): 解矩阵的方程组

4.1 原理解析

找到了锚点后，我们要计算它的速度向量 $\mathbf{v} = [u, v]^T$ 。根据光照不变性定理与一阶泰勒展开，我们能推导出超定方程组 $A\mathbf{v} = -b$ 。通过最小二乘法，两边同乘 A^T ，左边得到的矩阵 $A^T A$ ，正是我们刚才寻找角点时构建的结构张量矩阵 M ！方程组最终化简为：

$$\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} -\sum I_x I_t \\ -\sum I_y I_t \end{bmatrix} \quad (2)$$

4.2 C++ 克莱姆法则求解

既然矩阵相同，光流法实际上就是在解以该矩阵为系数的二元一次方程组。我们在代码中计算矩阵的行列式 $\det$ ，并用克莱姆法则直接求解出屏幕上那一抹赛博朋克的尾迹：

```
1 // 1. 计算行列式 det = ad - bc
2 double det = sum_Ixx * sum_Iyy - sum_Ixy * sum_Ixy;
3
4 // 物理意义：行列式为0代表特征值极小（白墙），方程无解，抛弃该点
5 if (abs(det) < 1e-5) return false;
6
7 // 2. 使用克莱姆法则求解瞬时速度 u, v
8 double u = (sum_Iyy * (-sum_Ixt) - sum_Ixy * (-sum_Iyt)) / det;
9 double v = (sum_Ixx * (-sum_Iyt) - sum_Ixy * (-sum_Ixt)) / det;
```